

T

ThinkAiSolutions

WhatsApp Business Platform — How the whole system works

Meta → Pinnacle (BSP) → **ThinkAiSolutions** → our clients

A complete walkthrough — business model, architecture, every screen, and the money/message data flows.

Generated for the ThinkAiSolutions reseller platform · Phase 1 (MVP).

Contents

1. **1 · What this platform is** — the reseller business model
2. **2 · The big picture** — system architecture at a glance
3. **3 · Who uses it** — roles & what each one sees
4. **4 · Logging in** — auth & how role/tenant are resolved
5. **5 · Reseller admin walkthrough** — onboarding a client, step by step
6. **6 · Tenant admin & agent walkthrough** — dashboard, templates, inbox
7. **7 · The wallet** — recharge, GST, and per-message billing
8. **8 · How a message is sent & billed** — the core data flow
9. **9 · How a reply comes back in** — inbound & the 24-hour window
10. **10 · Data model** — what is stored where
11. **11 · Deployment & security** — where it runs, how it's protected
12. **12 · Glossary**

About the screenshots: screens behind a login can't be auto-captured without credentials, so this doc uses **LIVE** real captures for public screens (login, WABA connect) and clear **ILLUSTRATION** mock-ups (drawn to match the real emerald/blue UI) for the rest, alongside diagrams of the internal flows.

1 · What this platform is

ThinkAiSolutions is a **multi-tenant WhatsApp Business API reseller platform**. WhatsApp messaging isn't sold directly by Meta to most businesses — it flows through a chain of providers. This platform's job is to let ThinkAiSolutions **become a reseller** in that chain: buy WhatsApp capacity wholesale and resell it to its own clients (clinics, pharmacies, labs, hotels, restaurants) under its own brand, dashboard, billing, and markup.

The supply chain — today vs. the goal

CURRENT (reseller-of-a-reseller)



THE GOAL (you ARE the reseller — replace Twincles)



The platform inserts ThinkAiSolutions as the branded reseller layer between the BSP and the end clients.

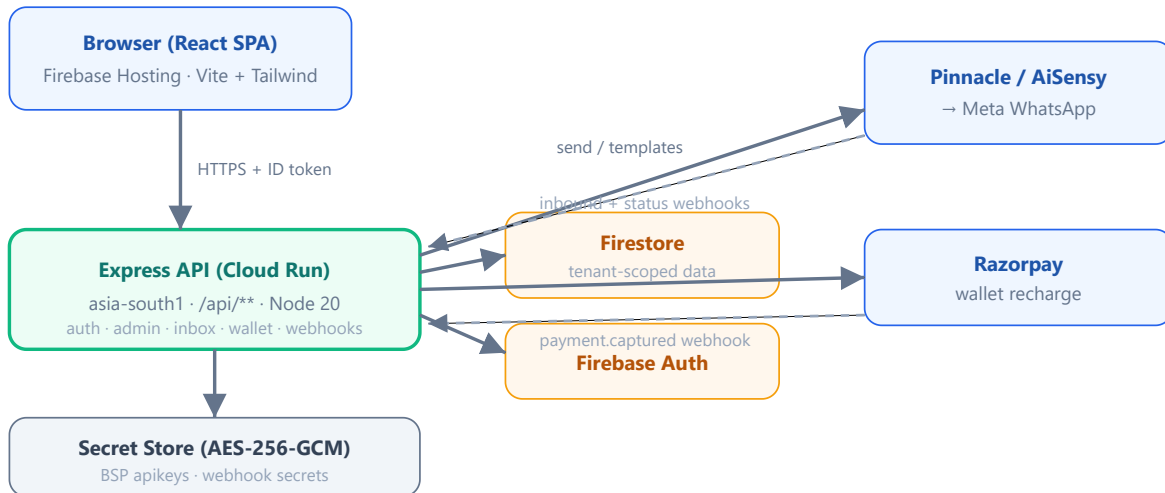
How ThinkAiSolutions makes money

- Meta charges **per message**, by category: **marketing** **utility** **authentication** are paid; **service** replies inside a 24-hour window are free.
- ThinkAiSolutions pays the BSP a wholesale **cost** rate, and sets a higher **charge** rate for each client. The difference is the **margin**.
- Clients pre-pay into a **wallet**. Every billable message is debited from that wallet automatically. When the wallet is empty, sending is blocked.
- All money is stored as **integer paise** (never floating-point rupees) to avoid rounding errors.

Multi-tenant from day one. Each client is a "tenant". Every piece of data — contacts, messages, wallet, pricing — is scoped to a `tenantId`, and one tenant can never see another's data.

2 · The big picture — system architecture

The platform is a small number of cleanly separated pieces. The browser only ever talks to **our own API**; it never touches the BSP or payment secrets directly.



Key rule: the browser only talks to the Express API. Apikeys, the Razorpay secret, and Firebase admin credentials never leave the server. Solid lines = outbound calls; dashed = inbound webhooks.

The three code packages (one repository)

Package	What it is	Runs / deploys to
<code>client</code>	React + Vite + Tailwind single-page app — everything the user sees.	Firebase Hosting (static)
<code>server</code>	Node + Express (TypeScript) — the API, billing, BSP calls, webhooks.	Google Cloud Run (asia-south1)
<code>shared</code>	Types, constants, and helpers (GST math, GSTIN validation) used by both sides so they never drift.	Compiled into both

BSP isolation: every call to a messaging provider goes through one interface (`BspProvider`) with one file per provider (`pinnacle.ts` , `aisensy.ts`). Swapping or adding a provider means writing one new file — nothing else changes.

3 · Who uses it — roles

There are exactly three roles. A user's role and tenant are stored as **custom claims** on their login token, so the app knows who they are the instant they sign in.

Role	Who	tenantId	Can do
reseller_admin	You (ThinkAiSolutions)	null (sees all)	Create & manage every tenant, set each tenant's pricing, view global usage / revenue / margin.
tenant_admin	Your client (e.g. a clinic owner)	their tenant	Manage their WABA, templates, inbox, wallet & recharge , and their agents — scoped to their tenant only.
agent	Client's staff	their tenant	Handle the inbox: read conversations, reply, send templates. No wallet access.

What each role lands on

RESELLER ADMIN → /ADMIN

- Tenants list
- New tenant (4-step wizard)
- Tenant detail
- Pricing editor
- Global usage & revenue

TENANT ADMIN / AGENT → /DASHBOARD

- Dashboard (metrics + charts)
- Inbox
- Templates
- Campaigns / Reports coming soon
- Wallet (admin only)

Routes are guarded: a signed-in user who tries to open a screen their role can't access is quietly redirected to their own home page. Not-signed-in users go to the login screen and are returned to where they were after signing in.

4 · Logging in

Everyone starts at the same login screen. There is no public sign-up — accounts are created for clients by the reseller admin (see §5).

What happens when you click "Sign in"

- 1** **Firebase Auth checks the password.** The browser sends email + password to Firebase Authentication and gets back a signed **ID token**.
- 2** **Role & tenant are read from the token.** The token carries custom claims (`role` , `tenantId`) that the server set when the account was created. No extra database lookup is needed.
- 3** **You're routed to your home.** Reseller admins go to `/admin` ; tenant admins and agents go to `/dashboard` .
- 4** **Every later API call carries the token.** The app attaches `Authorization: Bearer <token>` to each request; the server verifies it and enforces the role/tenant on every endpoint.

Why this is safe: the token is cryptographically signed by Firebase and verified on the server with the Firebase Admin

SDK on every request, including a check that the token hasn't been revoked.

5 · Reseller admin — onboarding a client

This is the most important admin journey: turning a new client into a live tenant who can send WhatsApp messages. It's a **4-step wizard** at `/admin/tenants/new`. Crucially, the tenant is **created at step 1** and the same `tenantId` is carried through steps 2–4 — so the admin never types an ID by hand.

T New tenant

① Tenant & billing ② Tenant admin ③ WhatsApp number ④ Pricing

The horizontal stepper at the top of every step. Completed steps turn green; the current step is highlighted.

Step 1 — Tenant & billing

Enter the client's name, optional plan, and billing details (legal name, **GSTIN**, state code, address). The GSTIN is validated before submit, and if you leave the state code blank it's derived automatically from the GSTIN — this drives the correct GST split on invoices later.

```
POST /api/admin/tenants → returns { tenantId } (also seeds the wallet at 0 paise)
```

Step 2 — Tenant admin login

Create the client's own login (name, email, temporary password). This user becomes a `tenant_admin` for the tenant created in step 1.

```
POST /api/admin/users → creates Firebase Auth user + sets role/tenant claims
```

Step 3 — Connect the WhatsApp number

This is the screen you asked about earlier. Pick the provider, enter the WhatsApp number and Meta identifiers, and paste the BSP apikey.

1 Tenant & billing — 2 Tenant admin — 3 WhatsApp number — 4 Pricing

Connect the tenant's WhatsApp number via Pinnacle. The apikey is stored securely and never shown again.

Provider
Pinnacle (PinBot)

Display name
Arjun CLinics

Phone number (E.164)
9391714623

WABA id (Meta)
Used for template fetch

phone_number_id (Meta)
Used for sending + webhooks

BSP apikey (write-only)
Pasted once, stored securely server-side

The key is encrypted server-side and never shown again. To rotate it, submit a new key.

Connect & continue

Back

LIVE Step 3 — connecting a WABA via Pinnacle. The **apikey is write-only**: pasted once, encrypted server-side, never shown again.

Field	Purpose
Provider	Which BSP this number runs on (Pinnacle or AiSensy). Switching changes the fields below.
Display name	Friendly label for the number.
Phone number (E.164)	The WhatsApp number — must include country code, e.g. <code>+919391714623</code> .
WABA id (Meta)	Used to fetch templates from Meta.
phone_number_id (Meta)	Used for sending and to route inbound webhooks back to this tenant.
BSP apikey write-only	The provider secret. Encrypted with AES-256-GCM and stored only as a reference; to change it you submit a new key.

POST /api/admin/wabas → stores WABA + encrypts apikey + registers the inbound webhook with the BSP

Real-world caution (from project notes): registering the webhook re-points inbound messages for that number to this platform. For a number with a live production bot, only do this with explicit intent to migrate, or inbound will be redirected away from the existing bot.

Step 4 — Pricing

Set the per-category **charge** rate (what the client pays) and optionally record your **cost** rate (what the BSP charges you). The form previews the **margin** live, colour-coded green/red.

PUT /api/admin/pricing/:tenantId → on success, lands on the tenant's detail page

After step 4 the client is fully live: they can log in, and once their wallet has a balance, send messages.

5 (cont.) · The rest of the admin console

Tenants list — </admin>

T

Tenants

+ New tenant

Name	Plan	Status	GSTIN
Arjun Clinics	starter	active	36ABCDE1234F1Z5
Mithra Medicare	—	active	36MITHR...

ILLUSTRATION A read-only table of every tenant, with a link to global usage and the "New tenant" button.

Pricing editor — </admin/pricing/:tenantId>

T

Pricing — Arjun Clinics

Category	Charge (₹/msg)	Our cost (₹/msg)	Margin
Marketing	0.99	0.78	+0.21
Utility	0.35	0.30	+0.05
Authentication	0.40	0.42	-0.02

ILLUSTRATION Charge – cost = margin, computed live. Cost rates are private to you (clients never see them).

Global usage & revenue — </admin/usage>

MESSAGES	REVENUE	COST	MARGIN
12,480	₹9,640	₹7,410	₹2,230

ILLUSTRATION Headline totals plus a per-tenant breakdown; click a tenant to drill into its detail page.

6 · Tenant admin & agent — daily use

Dashboard — /dashboard

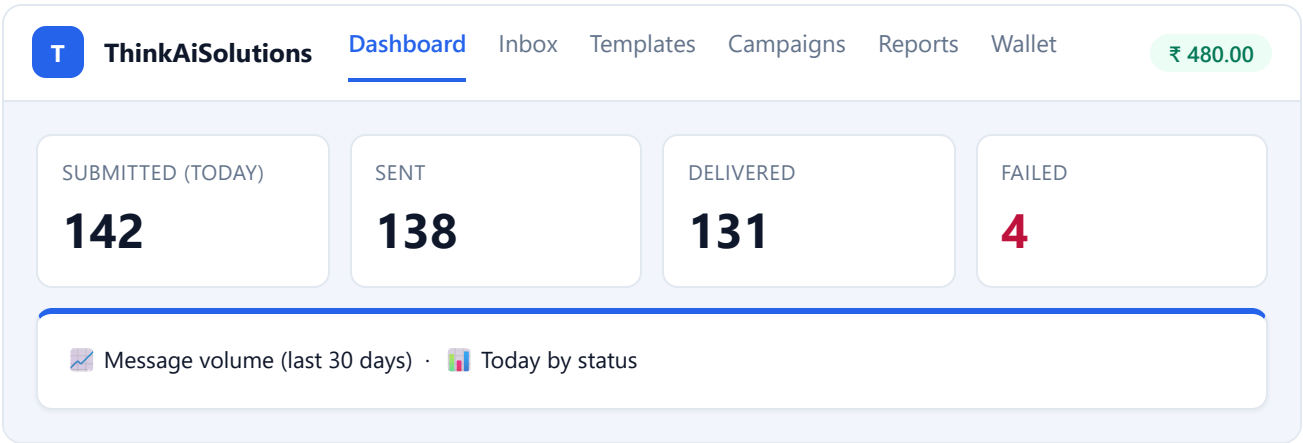


ILLUSTRATION Eight metric cards (today + last 30 days) and two charts. The header shows the live wallet balance and a logout menu.

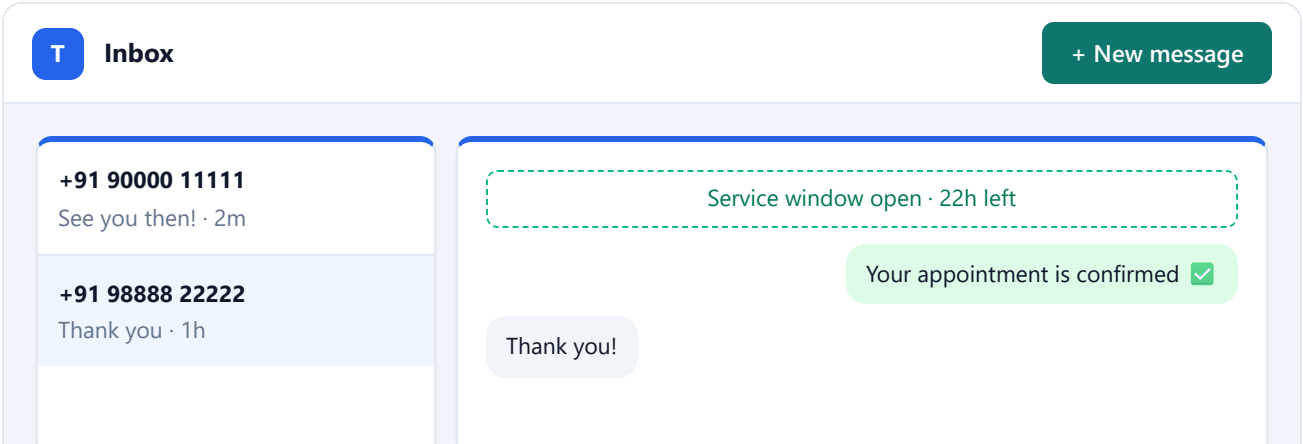
Templates — /templates

WhatsApp requires Meta-approved templates to start conversations. This screen lists the tenant's templates with status badges and a **"Sync from Pinnacle"** button that pulls the latest approved templates from the BSP.

Template	Category	Lang	Status
appointment_reminder	utility	en	approved
lab_report_ready	utility	en	approved
festival_offer	marketing	en	pending
otp_login	authentication	en	rejected

ILLUSTRATION Only **approved** templates can actually be sent.

Inbox — /inbox



Type a reply...

Send

Two panes: conversation list (left, auto-refreshing) and the open thread (right). The banner shows the **24-hour service window** status. Inside the window, agents can reply with free text (free). Outside it, the only way to message is an approved template (paid). The **+ New message** button opens a modal to send a template to any number — choose the template, fill its variables, send.

7 · The wallet — recharge & billing

Each tenant has one prepaid wallet (`tenant_admin` only — agents can't see it). The balance funds every billable message.

BALANCE

₹ 480.00

Recharge Refresh

RECHARGE PREVIEW

Credit	₹1,000.00
GST (18%)	₹180.00
You pay	₹1,180.00

ILLUSTRATION The balance card and the recharge modal's GST preview.

The GST rule (important): 18% GST is charged **once, at recharge, on top** of the credit. So if a client wants ₹1,000 of sending power, they pay ₹1,180 and the wallet is credited ₹1,000. Per-message debits afterwards are the **bare** rate with no further GST — this avoids charging GST twice.

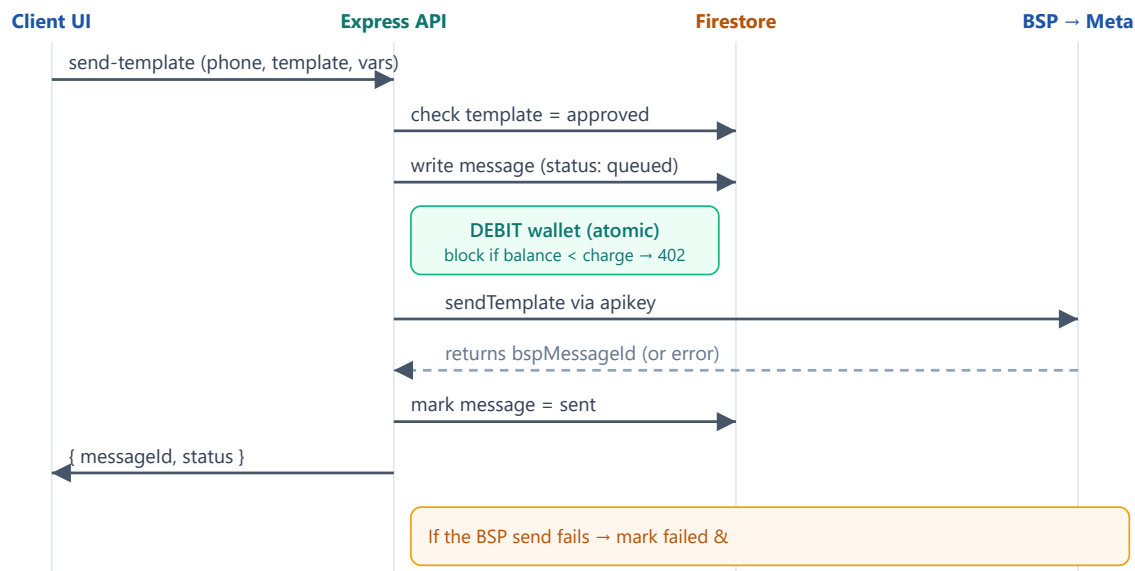
How recharge actually works

- 1 **Client enters an amount** (or taps a quick button: 500 / 1000 / 2500 / 5000). The modal previews credit + GST + total.
- 2 **Server creates a Razorpay order** for the total (credit + GST) and saves a `walletOrder` record with the exact amounts. `POST /api/wallet/recharge/order`
- 3 **Razorpay Checkout opens** in the browser; the client pays by card / UPI / netbanking.
- 4 **The browser does NOT credit the wallet.** It just shows "Payment received — balance will update shortly" and starts polling.
- 5 **Razorpay calls our webhook** (`payment.captured`). The server **verifies the signature**, looks up the saved order (amounts come from *our* record, never the webhook), **credits the wallet**, writes a transaction, and generates a **GST invoice** — all idempotently (a repeated webhook can't double-credit).
- 6 **The balance updates** and the transaction history shows the recharge.

Why credit only on the verified webhook? A browser can be tampered with; the signed server-to-server webhook is the only trustworthy confirmation that money actually arrived.

8 · How a message is sent & billed

This is the heart of the platform. When a tenant sends a template, the order of operations is deliberate: **debit first, send second, refund if the send fails** — so a client is never charged for a message that didn't go out, and can't send for free.



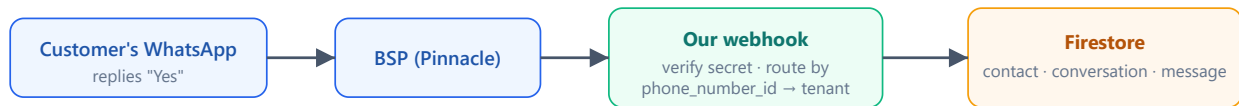
REFUND the debit (client pays nothing).

The rules baked in

- **Debit is atomic.** Reading the balance, checking it, decrementing it, and writing the ledger row all happen in one Firestore transaction — no double-spend, even under concurrent sends.
- **Insufficient balance blocks the send** with a clear `402 insufficient_funds` ; nothing is sent.
- **Idempotent by messageId.** Each debit is keyed to its message, so a retried request can't charge twice.
- **Free-text replies are different:** allowed only while the 24-hour window is open, and they cost nothing (no debit).
- **Delivery status** (sent → delivered → read, or failed) arrives later via the BSP webhook and updates the message — only ever moving forward, never backward.

9 · How a reply comes back in

When a customer replies on WhatsApp, the BSP sends it to our webhook. This both stores the message and **opens (or extends) the 24-hour service window**.



What the webhook writes (all idempotent by message id)

- ① upsert the contact · ② open/extend conversation 24h window · ③ store inbound message
The same endpoint also receives delivery-status updates for messages you sent.

The 24-hour service window — why it matters

- The window **opens only on an inbound message** from the customer, and lasts 24 hours.
- **While open:** agents can reply with free text, for free.
- **While closed:** the only way to reach the customer is an approved **template**, which is billed.
- This mirrors Meta's own rules and is why templates (and the wallet) matter for re-engaging customers.

Idempotency everywhere: webhooks can be delivered more than once. Every write is keyed (by message id, payment id, or event id) so duplicates are harmless — no double messages, no double charges.

10 · Data model — what is stored where

Everything lives in Firestore. Almost all data is nested under `tenants/{tenantId}/...` so isolation is structural. The browser never reads Firestore directly — it goes through the API — but the security rules still lock everything down as defence in depth.

Collection	Holds
<code>tenants/{id}</code>	Client profile: name, plan, status, billing (GSTIN, state, address).
<code>users/{uid}</code>	Each login's role, tenantId, name, email.
<code>tenants/{id}/wabas/{wabaId}</code>	Connected WhatsApp numbers + references to the encrypted apikey / webhook secret (never the secret itself).
<code>tenants/{id}/contacts</code>	Customers: phone, name, tags, opt-in status.
<code>tenants/{id}/templates</code>	Approved/pending templates synced from the BSP.
<code>tenants/{id}/conversations</code>	One thread per customer + the <code>windowExpiresAt</code> timestamp.
<code>tenants/{id}/messages</code>	Every message in/out: direction, status, category, <code>costPaise</code> , <code>bspMessageld</code> .
<code>tenants/{id}/wallet/current</code>	The single balance doc (<code>balancePaise</code>).
<code>tenants/{id}/walletTransactions</code>	The ledger: recharge / debit / refund, with balance-after.
<code>tenants/{id}/walletOrders</code>	Razorpay orders awaiting payment (server-only).
<code>pricing/{id}</code> & <code>pricingCost/{id}</code>	Charge rates (client-visible) and your cost rates (admin-only) per tenant.
<code>invoices/{paymentId}</code>	GST invoices with the CGST/SGST or IGST split.
<code>secrets/{ref}</code>	AES-256-GCM encrypted apikeys & webhook secrets. Deny-all to clients.
<code>processedEvents/{key}</code>	Webhook dedup markers (idempotency).

Authorization is O(1): the rules read `role` and `tenantId` straight off the signed token — no extra database lookups — so a tenant member can only ever read their own subtree, and reseller-only data (cost rates, secrets, orders) is never exposed to tenants.

11 · Deployment & security

Where it runs

Piece	Hosted on	How it ships
Client (SPA)	Firebase Hosting	Vite build → <code>client/dist</code> → <code>firebase deploy</code>
API server	Cloud Run (asia-south1), service <code>thinkai-server</code>	Dockerfile → <code>gcloud run deploy --source .</code>
Routing	Firebase Hosting rewrite	<code>/api/**</code> → Cloud Run; everything else → the SPA
Data / Auth	Firebase (Firestore + Auth)	Rules & indexes deployed from the repo

The security model in one view

- **Secrets stay server-side.** BSP apikeys, the Razorpay secret, the encryption key, and Firebase admin credentials live only on the server — never in the browser bundle, never in a Firestore document in plaintext.
- **The browser bundle is public by design.** Only safe values ship to it (the Firebase web config and the Razorpay *public* key id), and access is gated by Auth + rules.
- **Webhooks are verified before trust.** Razorpay by HMAC-SHA256 signature; the BSP by a constant-time secret-header compare. Unsigned/forged calls are rejected.
- **Money is paise, and atomic.** Integer paise throughout; all balance changes run in Firestore transactions; all credits/debits are idempotent so retries are safe.
- **Tenant isolation is enforced server-side.** A tenant's scope comes from their token, never from the request body — they can't smuggle another tenant's id.
- **Apikeys are encrypted at rest** (AES-256-GCM) and only decrypted in memory at the moment of a send.

Current status & what's next

The app is **live** — client on Firebase Hosting, server on Cloud Run (asia-south1). Login, tenant onboarding, WABA connect, template send, inbox, the 24-hour window, and wallet/Razorpay billing are all built (Phase 1 / MVP).

Deliberately out of scope for now (later phases): a template builder & approval flow, broadcast campaigns, a no-code chatbot, contact import/segments, Meta embedded signup, vertical packs (Clinics / Restaurants), AI chat/voice, and white-label branding.

12 · Glossary

Term	Meaning
BSP	Business Solution Provider — a company (like Pinnacle) authorised to sell access to Meta's WhatsApp API.
Reseller	A layer that buys from a BSP and resells to end clients. ThinkAiSolutions aims to be this layer (replacing Twincles).
Tenant	One client business on the platform. All their data is isolated under one <code>tenantId</code> .
WABA	WhatsApp Business Account — the Meta object a phone number lives under.
Template	A pre-approved message format required to start (or re-open) a conversation. Billable.
Service window	The 24-hour period after a customer messages you, during which free-text replies are free.
Paise	1/100 of a rupee. All money is stored as whole paise to avoid float errors.
GST	Indian 18% tax, charged once at wallet recharge; split into CGST+SGST (same state) or IGST (different state).
Idempotent	An operation that's safe to repeat — running it twice has the same effect as once (used for webhooks & billing).
Custom claims	Role & tenant data baked into the login token so the app knows who you are without extra lookups.

That's the whole system. A client logs in → (admin onboarded them with a WABA, pricing, and a login) → they top up a wallet → send approved templates that are billed per message → customers reply within a 24-hour window where free-text is free → everything is stored per-tenant, with secrets encrypted and every webhook verified.